REGISTRADORES E O FORMATO DA INSTRUÇÃO · ENCONTRO 2 · LIGA PROTOFPGA

Folha de atividade

Nomes da dupla: _____ *Data:* __ / ___

Parte A — Atividade em aula (em duplas)

Trabalhem na mesma máquina, com o arquivo `aula2_campos.asm` aberto no MARS. A **folha de consulta**, na última página, tem todas as tabelas de que vocês vão precisar. Se travarem, chamem.

A1 — Observar (5 min)

1. Abram o arquivo `aula2_campos.asm` no MARS e cliquem em Assemble.

2. No painel de registradores, a coluna Number mostra o número de cada um. Anotem:

`$zero = ___` *`$at = __`* *`$v0 = __`* *`$t0 = __`* *`$t7 = ___`*

`$s0 = ___` *`$t8 = __`* *`$sp = __`* *`$ra = ___`*

3. Na janela Text Segment, copiem a coluna Code das linhas do `add` e do `sub`. Circulem os dígitos hexadecimais que são diferentes entre os dois.

`add: 0x_____` *`sub: 0x_____`*

A2 — Decompor (12 min)

4. Escrevam em binário o código do `sub` e separem os 32 bits nos campos do formato R.

`binário: _____ _____ _____ _____`

Linha de cima: os bits de cada campo. Linha de baixo: o valor em decimal e o que ele significa.

op	rs	rt	rd	shamt	funct

5. Comparando com o `add`, decomposto no slide, quais campos mudaram? Por que mudou mais de um, se a instrução só trocou de operação?

6. Copiem da coluna Code o código de `addi $s0, $t2, -1` e decomponham no formato I. Qual campo guarda o destino? Quanto vale o imediato em decimal?

`código: 0x_____` *`binário: _____ _____ _____ _____`*

Linha de cima: os bits de cada campo. Linha de baixo: o valor em decimal e o que ele significa.

op	rs	rt	imediato

A3 — Montar à mão (10 min)

7. Sem usar o MARS, escrevam os campos, os 32 bits e o hexadecimal da instrução `or $t6, $t4, $t5`.

Linha de cima: os bits de cada campo. Linha de baixo: o valor em decimal e o que ele significa.

op	rs	rt	rd	shamt	funct

`32 bits: _____ _____ _____ _____ hex: 0x_____`

8. Acrescentem essa linha ao programa, logo antes de `li $v0, 10`. Montem de novo e confiram na coluna Code. Acertaram? Se não, qual campo estava errado?

A4 — Provocar (8 min)

9. Antes de executar: quanto vai valer `$zero` depois da linha `li $zero, 7`? Escrevam a previsão, executem com Step e confiram. Expliquem o que aconteceu.

`previsão: $zero = _____ resultado: $zero = _____`

10. Procurem `li $s1, 100000` na janela Text Segment. Quantas linhas ele ocupou? Qual registrador aparece na coluna Basic sem que vocês o tenham escrito? Quanto ele vale ao final?

11. Abram `aula2_transbordo.asm`. Antes de executar, escrevam a previsão. Depois executem com Step e anotem o que aconteceu na última soma.

`previsão: $t2 = _____ $t3 = _____`

`resultado: $t2 = _____ $t3 = _____`

Parte B — Desafio da semana

O programa escondido

Uma dupla deixou na memória do simulador só os códigos abaixo — o texto do programa se perdeu. Recupere-o.

(1) Decomponha cada código e reescreva o programa em assembly, com os apelidos dos registradores. **(2)** Preveja os valores finais de `$t0`, `$t1`, `$t2`, `$s0` e `$s1`. **(3)** Digite o programa no MARS, acrescente a fórmula de encerramento, monte e confira: as cinco primeiras linhas da coluna Code precisam mostrar exatamente estes números.

Pista: um dos valores é negativo, e o último resultado vai exigir olhar os bits.

Endereço	Code	Instrução em assembly
0x00400000	0x24080007	
0x00400004	0x2409FFFD	
0x00400008	0x01095022	
0x0040000C	0x01488020	
0x00400010	0x01498824	

Previsão:

`$t0 = _____` `$t1 = ____` `$t2 = ____` `$s0 = ____` `$s1 = _____`

Espaço para as decomposições:

Parte C — Exercícios de revisão

Onze perguntas, para fazer em casa. As que pedem o MARS estão marcadas. Tentem responder tudo antes de abrir o gabarito — conferir cedo demais desperdiça o exercício.

1. Por que o banco de registradores do MIPS tem exatamente 32 registradores?

2. Verdadeiro ou falso: o processador impede que se use `$s0` para guardar um valor temporário. Justifique.

3. Suponha `$t0` = 4 e `$t1` = 6. Quanto vale `$zero` depois de `add $zero, $t0, $t1`? Por quê?

4. Em `add $t2, $t0, $t1`, o destino é o primeiro registrador escrito. Em que campo, e em quais bits dos 32, ele aparece no código de máquina?

5. `add` e `sub` têm o mesmo op. Como o processador sabe qual das duas executar?

6. No encontro 1, trocar 12 por 20 em `li $t0, 12` mudou o código de `0x2408000C` para `0x24080014`. Usando o formato I, diga exatamente qual campo mudou e qual campo indica o registrador de destino.

7. Qual é o maior e o menor valor que cabem no imediato de `addi`? Por quê?

8. Escreva −5 em complemento de dois com 16 bits, em binário e em hexadecimal. Mostre as etapas.

9. Decomponha `0x012A8824` e escreva a instrução em assembly, usando os apelidos dos registradores.

10. (MARS) Monte à mão o código de `sub $s2, $s0, $t0`: escreva os campos, os 32 bits e o hexadecimal. Depois confira no MARS.

11. (MARS) Execute `aula2_transbordo.asm` com Step. Explique por que `addu` e `add`, com as mesmas parcelas, terminam de jeitos diferentes. Relacione com o quadro sobre `addi` e `addiu` da apostila do encontro 1.

FOLHA DE CONSULTA · GUARDE PARA OS PRÓXIMOS ENCONTROS

Os formatos

R · op	rs	rt	rd	shamt	funct
31–26	25–21	20–16	15–11 · destino	10–6	5–0

I · op	rs	rt	imediato
31–26	25–21	20–16 · destino	15–0

J · op	endereço
31–26	25–0

Ordem no texto: R → `op rd, rs, rt` · I → `op rt, rs, imediato`. **Qual formato?** op = `000000` → R; `000010`/`000011` → J; outro → I. **Complemento de dois:** −x = inverta os bits de x e some 1; imediato de −32 768 a 32 767.

Os registradores

nº	bits	nome	nº	bits	nome	nº	bits	nome	nº	bits	nome
0	00000	$zero	8	01000	$t0	16	10000	$s0	24	11000	$t8
1	00001	$at	9	01001	$t1	17	10001	$s1	25	11001	$t9
2	00010	$v0	10	01010	$t2	18	10010	$s2	26	11010	$k0
3	00011	$v1	11	01011	$t3	19	10011	$s3	27	11011	$k1
4	00100	$a0	12	01100	$t4	20	10100	$s4	28	11100	$gp
5	00101	$a1	13	01101	$t5	21	10101	$s5	29	11101	$sp
6	00110	$a2	14	01110	$t6	22	10110	$s6	30	11110	$fp
7	00111	$a3	15	01111	$t7	23	10111	$s7	31	11111	$ra

Os códigos

R (op = 0)	funct	dec		I e J	op	dec
add	100000	32		addi	001000	8
addu	100001	33		addiu	001001	9
sub	100010	34		ori	001101	13
subu	100011	35		lui	001111	15
and	100100	36		lw	100011	35
or	100101	37		sw	101011	43
xor	100110	38		beq	000100	4
nor	100111	39		bne	000101	5
slt	101010	42		j	000010	2
syscall	001100	12		jal	000011	3

Hexadecimal ↔ binário

0 0000	**1** 0001	**2** 0010	**3** 0011	**4** 0100	**5** 0101	**6** 0110	**7** 0111
8 1000	**9** 1001	**A** 1010	**B** 1011	**C** 1100	**D** 1101	**E** 1110	**F** 1111